PaaS4Bat Dashboard — Technical Documentation

1. Introduction

PaaS4Bat is a fullstack dashboard for predicting battery capacity degradation and remaining useful life from early-cycle battery cycling data. The goal is to provide a clean web interface for uploading CSV files, running prediction through a mock API, visualizing degradation trends, comparing two batteries, and exporting results for reporting.

Goals and Scope

The application supports:

- Mock login with username and password.
- Protected dashboard routes.
- CSV upload with validation.
- Sample data prediction.
- Capacity degradation chart with confidence interval.
- Cycle life, capacity at cycle 500, and capacity at cycle 1000 metrics.
- Degradation mechanism breakdown.
- Battery comparison view.
- CSV/PDF export.
- Docker and Scaleway deployment.

Out of scope: real ML model training, production authentication, database persistence, and user role management.

2. Architecture Overview

The project uses a decoupled frontend and backend architecture.

```
Browser
  → React + Vite frontend
  → POST /api/predict
  → FastAPI backend
  → CSV validation + mock prediction
  → JSON prediction response
  → Dashboard visualization
```

Runtime Flow

1. User opens the app.
2. If not logged in, the user is redirected to `/login`.
3. User logs in with username and password.
4. User uploads a CSV file or selects sample data.
5. Frontend sends the CSV as `multipart/form-data` to `/api/predict`.
6. Backend validates the CSV and generates mock prediction results.
7. Frontend stores the prediction in Zustand.
8. Dashboard renders charts, metrics, explanation, export actions, and comparison option.

Main Structure

```
backend/
  app/
    api/predict.py
    services/csv_parser.py
    services/mock_predictor.py
    main.py
    schemas.py

frontend/
  src/
    components/
    hooks/
    lib/
    pages/
    stores/
    types/
  public/sample/
```

Page	Responsibility
`LoginPage.tsx`	Mock login form
`UploadPage.tsx`	CSV/sample upload
`DashboardPage.tsx`	Prediction visualization
`ComparePage.tsx`	Battery comparison

3. Technology Stack

Layer	Technology	Reason
Frontend	React 18 + Vite	Fast dashboard development and build performance
Language	TypeScript	Safer API and UI state handling
Styling	Tailwind CSS	Responsive enterprise-style UI
Charts	Chart.js	Line chart, confidence interval, tooltips
State	Zustand	Lightweight global state and persistence
Async/API	TanStack Query	Mutation loading/error handling
Backend	FastAPI	Simple Python API with Swagger docs
Validation	Pydantic + CSV parser	Structured API and CSV validation
Export	jsPDF + html2canvas	PDF report generation
Deployment	Docker + Scaleway	Reproducible cloud deployment

This stack matches the case study direction and keeps the project easy to review, run, and deploy.

4. Implementation Details

Authentication and Route Protection

The app implements mock authentication. The login page has username and password fields and a login button. Any non-empty credentials are accepted. Protected routes check `isAuthenticated` from Zustand. If the user is not logged in, `/`, `/dashboard`, and `/compare` redirect to `/login`. The logout button clears authentication and returns the user to the login page.

Upload and Prediction Flow

Both manual upload and sample data use the same backend endpoint:

```
POST /api/predict
```

The request is sent as `multipart/form-data` with:

Field	Type	Required
`file`	CSV file	Yes
`battery_label`	string	No

Sample CSV data is fetched from `public/sample`, converted to a `File`, and submitted through the same upload function as a real user file.

CSV Format

Required columns: `cycle`, `capacity_ah`.

Optional columns: `voltage_avg`, `temperature_c`, `coulomb_efficiency`.

```
cycle,capacity_ah,voltage_avg,temperature_c,coulomb_efficiency
1,2.05,3.62,24.5,0.998
2,2.04,3.61,24.8,0.997
```

Dashboard and Comparison

The dashboard displays the capacity degradation curve, confidence interval, cycle life, capacity at 500/1000 cycles, degradation breakdown, explanation text, export actions, and comparison action.

The compare page uses Battery 1 from the dashboard and lets users add Battery 2 by upload or sample data. The page overlays both degradation curves and displays metrics side-by-side. When starting a new comparison, the previous Battery 2 result is cleared to avoid stale data.

5. UI/UX Decisions

The UI is designed as a clean enterprise dashboard. The main goals are clarity, minimal distractions, and easy interpretation of prediction results.

Key decisions:

- Login is the first screen for unauthenticated users.
- Upload page focuses only on CSV input and sample data.
- Dashboard uses a top-down structure: chart, metrics, breakdown, explanation, actions.
- Only requirement-related actions are shown: upload, compare, export, logout.
- Error messages are displayed inline and written for users, not developers.
- Loading states are shown during prediction.
- Layout supports desktop and tablet widths with responsive cards and wrapping actions.

6. API Integration

Local and Production Base URL

In local development, the frontend calls `/api/*` , and Vite proxies requests to `http://localhost:8000` .

In production, the frontend is built with:

```
VITE_API_URL=https://<backend-url>
```

Endpoints

Method	Endpoint	Description
GET	`/api/health`	Health check

Method	Endpoint	Description
POST	`/api/predict`	CSV file prediction

Prediction Response

```
{
  "prediction_id": "pred_a1b2c3d4",
  "cycle_life": 1247,
  "capacity_at_500": 92.3,
  "capacity_at_1000": 84.1,
  "confidence_interval": [81.5, 86.7],
  "degradation_breakdown": {
    "electrical": 25,
    "mechanical": 30,
    "chemical": 35,
    "coulombic": 10
  },
  "capacity_curve": [
    { "cycle": 0, "capacity": 100, "lower": 100, "upper": 100 }
  ]
}
```

The frontend handles invalid file type, missing columns, API validation errors, unavailable backend, failed sample loading, and export errors with user-friendly messages.

7. Deployment

The deployment target is Scaleway using Container Registry and Serverless Containers.

```
Scaleway Container Registry
  → paas4bat-api:latest
  → paas4bat-web:latest
  → Serverless Containers
     - Backend: FastAPI, port 8000
     - Frontend: Nginx, port 80
```

Backend

```
docker build --platform=linux/amd64 \
  -t rg.fr-par.scw.cloud/<namespace>/paas4bat-api:latest \
  ./backend

docker push rg.fr-par.scw.cloud/<namespace>/paas4bat-api:latest
```

Scaleway settings:

```
Name: paas4bat-api
Port: 8000
Environment:
  CORS_ORIGINS=*
```

After the frontend URL is available, update `CORS_ORIGINS` to `https://<frontend-url>`.

Frontend

```
docker build --platform=linux/amd64 \
  -t rg.fr-par.scw.cloud/<namespace>/paas4bat-web:latest \
  --build-arg VITE_API_URL=https://<backend-url> \
  ./frontend

docker push rg.fr-par.scw.cloud/<namespace>/paas4bat-web:latest
```

Scaleway settings:

```
Name: paas4bat-web
Port: 80
```

Verification and Evidence

Verify backend at `https://<backend-url>/api/health` and frontend at `https://<frontend-url>`. Confirm login, upload, dashboard, compare, export, and logout.

Submission evidence should include screenshots of registry, backend container, frontend container, environment variables, live frontend, and backend health/API docs.

8. Testing

Automated Checks

Backend:

```
cd backend
python -m pytest -q
```

Frontend:

```
cd frontend
npm run build
npm run lint
```

Required Manual Test Cases

ID	Test	Expected Result	Status
T-01	Upload valid CSV	Prediction displays correctly	Pass
T-02	Upload invalid image file	Error message shown	Pass
T-03	Upload CSV with missing columns	Graceful validation error	Pass
T-04	Use Sample Data	Dashboard populates with mock data	Pass
T-05	Tablet responsive test	Layout adapts, no horizontal scroll	Pass
T-06	Loading state	Spinner/skeleton appears during API call	Pass
T-07	API error handling	User-friendly error/fallback shown	Pass

ID	Test	Expected Result	Status
T-08	Deployed URL accessible	Public URL works	Pass
T-09	Chart interactivity	Tooltip values display on hover	Pass
T-10	Export CSV/PDF	Files download correctly	Pass

Authentication Checks

Test	Expected Result
Open `/dashboard` without login	Redirects to `/login`
Empty login submit	Validation message shown
Valid mock login	User enters app
Logout	User returns to login page
Refresh after login	Session and prediction remain available

9. Challenges & Solutions

Challenge	Solution
Sample data used a different JSON endpoint	Converted sample CSV into a `File` and sent it to `/api/predict`
Preventing dashboard access before login	Added protected route logic
Keeping results after refresh	Persisted auth and prediction state with Zustand
Avoiding stale compare data	Reset Battery 2 when starting a new comparison
Apple Silicon deployment issue	Built Docker images with `--platform=linux/amd64`

Challenge	Solution
Nginx service name issue on Scaleway	Used `VITE_API_URL` for production backend URL
CSV validation	Backend validates file structure and required columns

10. Future Improvements

With more time, the project could add: real ML inference, production authentication, role-based access, database-backed prediction history, automated Playwright E2E tests, GitHub Actions CI/CD, monitoring, advanced chart interactions, model versioning, and audit logs.

11. Appendix

Local Development

Backend:

```
cd backend
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python -m uvicorn app.main:app --reload --port 8000
```

Windows activation:

```
.venv\Scripts\activate
```

Frontend:

```
cd frontend
npm install
npm run dev
```

Open `http://localhost:5173`.

Docker Local Run

```
docker compose up --build
```

Frontend: `http://localhost:8080`
Backend: `http://localhost:8000`

Documentation Files

```
README.md
docs/api.md
docs/testing.md
docs/technical-report.md
```

Submission Checklist

- Complete source code.
- README with setup and deployment instructions.
- Technical documentation exported as PDF.
- API documentation.
- Testing documentation.
- Minimum 5 screenshots.
- Scaleway deployment evidence.
- Public live URL.
- No `node_modules`.
- No `.env` secrets.
- No large binary files.